

Part 1 — Baseline Model

for the Architecture I built a MLP with 2 hidden layers like the TP want : $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$. The input is 784 because we have 28x28 image, and the output is 10 because we have 10 clothing classes. Counting the parameters of the 3 Linear layers gives 235,146 adjustable parameters total (confirmed by thecheck cell).

for the loss function I used `nn.CrossEntropyLoss()` and not MSE, because this is a classification problem (predicting a category, not a continuous value). in the CM1 we can see that cross-entropy is built to penalize the model more when it is confidently wrong, MSE is made for regression tasks where the output is a real number.

`model.train` permit to turn on the training mode, so activate the Dropout and the BatchNorm layers. And `model.eval()` permit to turn off the training mode.

in the Training configuration part. I trained with Adam optimizer, $lr=1e-3$, and no regularization here, `batch_size=64`, for 15 epochs. Adam was choosen as baseline because the CM2 show that it adapts the learning rate per parameter automatically, so it is more robust than plain SGD without needing to tune the lr manually first.

For the resultat we obtained 94.11% train accuracy and 88.78% validation accuracy. so the train-val gap is 5.33 points.

The gap keeps growing over training (from -2.89 at epoch 1 to 2.85 at epoch 10 and 5.33 at epoch 15), and val loss stops improving after epoch 5 (0.315) and drifts up to 0.386 at epoch 15, while train loss keeps dropping (0.27 -> 0.15).

Part 2 — Experiment 1: Model Architecture

Result Table :

Config	Train acc (ep15)	Val acc (ep15)	Gap	Test acc
Micro [4]	83.24%	82.42%	0.82	82.18%
Small [64]	92.37%	88.28%	4.09	87.90%
Baseline [256, 128]	93.90%	89.00%	4.90	88.32%
Large [512, 256]	94.33%	88.93%	5.40	88.68%

When I ran the cell, each configuration doesn't have the same excution time. Larger models take longer. Firstable, the Micro model is very quick, he have a low Gap dans his val-acc is correct.

A small gap doesn't always mean good generalization: Micro has the smallest gap (0.82) but also the lowest train accuracy (83.24%), meaning it's too small to even fit the training data well, this is underfitting, not good generalization.

Based on this, I would recommend the Small [64] or Baseline [256,128] architecture: val accuracy already plateaus around 88-89% starting from Small, and Large adds nothing over Baseline (88.93% vs 89.00%) despite having more than double the parameters, so the extra capacity mostly goes into overfitting instead of real improvement.

Part 2 — Experiment 2A: Learning Rate (SGD)

LR	Val acc (ep15)	Test acc
1e-5	33.65%	33.88%
1e-3	81.65%	80.49%
0.1	89.12%	88.18%

With $lr=1e-5$ the model almost don't learn: after 15 epochs val acc is just 33.65% (random classification would be 10% for 10 classes), the updates are too small to move the weights (CM2, too low lr). With $lr=1e-3$ the model learns but slowly (60% at epoch 1 to 81.65% at epoch 15) and is still a little bit far from optimal. $lr=0.1$ is the best one: 83% val acc from epoch 1 and it stays around 87-89%, so it converged fast. The cost of a bad lr is high, $lr=1e-3$ loses about 8 pourcent of test acc compared to $lr=0.1$ (80.49% vs 88.18%). So SGD is very sensitive to the learning rate, which motivates adaptive optimizers like Adam that we saw in CM2

Part 2 — Experiment 2B: Optimiser Comparison

Configuration	Learning Rate	Best val acc	Epochs to reach best	Test acc
SGD best	0.1	89.13%	12	88.20%
Adam best	1e-3	89.27%	14	88.71%

Best lr is 0.1 for SGD and $1e-3$ for Adam, they have almost the same best val acc (89.13% vs 89.27%). We can also see that the best lr Adam is a bit faster, it reaches 88% val acc at epoch 3 against epoch 7 for SGD, I think that is coherent with the adaptive step per parameter of Adam. With the same $lr=1e-3$ the difference is big, Adam gets 89.01% val acc but only 81.60% for SGB. And Adam with $lr=0.1$ is random (10.00% test acc) while $lr=0.1$ is the best one for SGD. To conclude, a good lr for one optimizer is not good for another one, the lr must be tuned for each optimizer, and Adam is more convenient because its default $lr=1e-3$ works directly.

Part 2 — Experiment 3: Regularisation

Configuration	Train (ep20)	Val (ep20)	Gap	Test acc	Epoch of best val acc
No regularisation	95.36%	89.06%	6.30	88.60%	15
Dropout 0.3	90.58%	89.46%	1.12	88.30%	18

Configuration	Train (ep20)	Val (ep20)	Gap	Test acc	Epoch of best val acc
Weight decay 1e-3	90.58%	87.98%	2.60	87.48%	17
Dropout + BN	91.53%	89.42%	2.11	88.89%	15

Without regularisation the model is very strong on train (95.36%) but the val stay around 89%, so the gap is very big (6.30), it's the overfitting. Dropout randomly turn off 30% of the neurons at each training step, so the network can't rely on one path and it learns more redundant features, it works on the activations.

Weight decay adds a penalty to the loss, so the big weights are punished and the model is pushed to simpler functions (CM2), it works directly on the weights. Dropout 0.3 has the smallest gap (1.12) and Dropout + BN has the best test acc (88.89%) and best val acc (89.72%), but at epoch 20 their val acc is almost the same (89.46% vs 89.42%), so the difference is small because val acc moves about ± 0.5 between epochs. Also the train acc with dropout is computed with dropout on, so the gap is a bit flattering. Weight decay is the worst one, the gap is smaller (2.60) but only because the train goes down (90.58%), the val doesn't improve (87.98%) and it is very unstable (86.69% at epoch 11), I think $\lambda=1e-3$ is too strong here. The Dropout + BN curve is the smoothest one, BatchNorm stabilizes the training (CM2).

For the epoch of best val acc, Dropout (18) and weight decay (17) need a bit more epochs than no regularisation (15). It makes sense because regularisation slows down the train (Dropout train acc is still going up at epoch 20), so the val keeps improving longer, while the model without regularisation reaches its plateau early and then only overfits. Dropout + BN is the exception (epoch 15), I think because BatchNorm speeds up the convergence (CM2). But val acc is noisy, so 15 vs 17-18 is only a tendency. So regularisation trades some training speed against a better generalisation.